High Performance Computing

Tutorial Series No. 01: Study Guide and Answers

دليل الدراسة وحلول سلسلة التمارين رقم01

This TD has questions that were not in the first slides (walls, FSB/QPI, ILP, SIMT). They are explained here from general computer architecture knowledge, so check them against your professor's notes.

هذا التمرين فيه أسئلة لم تكن في الشرائح الأولى (الجدران، FSB/QPI، ILP ، SIMT). تم شرحها من معرفة عامة بالمعمارية، فراجعها مع ملاحظات الأستاذ.

Part A: New concepts in this TD | المفاهيم الجديدة

The 3 walls | الثلاثة الجدران

Wall / الجدار	English	العربية
Power wall	Higher clock needs higher voltage, so power grows fast ($P \approx C \cdot V^2 \cdot f$, and V rises with f). The chip overheats.	رفع التردد يتطلب رفع الجهد، فتزداد الطاقة بسرعة ($P \approx C \cdot V^2 \cdot f$). الشريحة تسخن.
Memory wall	CPU speed grew much faster than RAM speed, so the CPU waits for data.	سرعة المعالج زادت أسرع بكثير من سرعة الذاكرة، فينتظر المعالج البيانات.
ILP wall	A single program has few independent instructions to run at once, so extra pipeline and superscalar hardware gives diminishing returns.	البرنامج الواحد فيه تعليمات مستقلة قليلة، فالعتاد الإضافي يعطي مكاسب متناقصة.

Most important: the power wall. It is a physical limit, and it is what directly forced the move to multicore. The other two can be partly reduced with caches and smarter design.

الأهم: جدار الطاقة. هو حد فيزيائي وهو الذي أجبر الصناعة على الأنوية المتعددة، أما الآخران فيمكن تخفيفهما بالذاكرة المخبئية والتصميم الذكي.

SIMD vs SIMT | الفرق

- **SIMD:** one instruction works on a wide vector register (e.g. AVX). The programmer or compiler vectorizes the code.
  - **SIMD:** تعليمة واحدة على سجل متجهي عريض، والمبرمج أو المترجم يقوم بالتوجيه.
- **SIMT:** you write code for one thread. Hardware groups threads into warps (32 threads on NVIDIA) running in lockstep. Each thread has its own registers and can diverge (masked).
  - **SIMT:** تكتب كوداً لخيط واحد، والعتاد يجمع الخيوط في warps (32 خيطاً في NVIDIA) تعمل بتزامن، ولكل خيط سجلاته وقد ينحرف عن الباقي.

Formulas | القوانين

```
Speedup      S = T1 / TT            Efficiency  E = S / N
Amdahl       S(N) = 1 / ( s + (1-s)/N )       max = 1/s
Gustafson    S(N) = N - s(N-1)   =   s + (1-s)N
N for target speedup X (Amdahl):   N = (1-s) / (1/X - s)
```

Part B: Answers | الإجابات

Exercise 1: The need for parallelism

1. Definitions | التعريفات

- **Transistor:** a tiny semiconductor switch (on = 1, off = 0). It is the basic building block of all digital chips.

- مفتاح صغير من أشباه الموصلات) تشغيل1 = ، إيقاف(0 = ، وهو اللبنة الأساسية للشرائح.
- **Integrated Circuit (IC):** many transistors and connections built on one silicon chip.
- عدد كبير من الترانزستورات والوصلات على شريحة سيليكون واحدة.
- **Microprocessor:** a CPU (ALU, control unit, registers) on one IC. It needs external memory and I/O.
- وحدة معالجة مركزية على شريحة واحدة، وتحتاج ذاكرة وإدخال/إخراج خارجيين.
- **Microcontroller:** CPU + memory (RAM/Flash) + I/O peripherals on one chip, for embedded systems (e.g. Arduino/ATmega).
- معالج + ذاكرة + منافذ على شريحة واحدة للأنظمة المدمجة.

2. Lithography | الليثوغرافيا

It is the process of printing the circuit pattern onto a silicon wafer with light through a mask (now EUV light). Better lithography means smaller transistors. Smaller transistors mean more per chip (**Moore's Law**) and, historically, lower power per transistor and higher frequency (**Dennard Scaling**). So lithography is the manufacturing technology that made both laws possible.

هي عملية طباعة نمط الدارة على رقاقة السيليكون بالضوء عبر قناع) اليوم ضوء .(EUV ليثوغرافيا أدق تعني ترانزستورات أصغر، أي عدداً أكبر في الشريحة) **قانون موور** (وطاقة أقل لكل ترانزستور وتردداً أعلى) **دينارد** .(فهي التقنية التي جعلت القانونين ممكنين.

3. Moore and Dennard, and 2005 | 2005 وعام وديناردد موور

Moore: transistor count doubles about every 2 years. **Dennard:** when transistors shrink, voltage and current shrink too, so power density stays constant and frequency can rise. Around **2005 Dennard Scaling ended** (leakage current, and voltage can't drop further). **Moore's Law continued**, but more slowly. Result: we get more transistors but can't make them faster, so we use them for **more cores**.

موور :عدد الترانزستورات يتضاعف كل سنتين تقريباً .**دينارد :**عند تصغير الترانزستور ينخفض الجهد والتيار فتبقى كثافة الطاقة ثابتة ويمكن رفع التردد .حوالي **2005 انتهى دينارد)** تسرب التيار وعدم إمكانية خفض الجهد أكثر .(**استمر موور** لكن ببطء .النتيجة :ترانزستورات أكثر لكن ليست أسرع، فنستعملها في **أنوية أكثر**.

4. Clock, heat, noise, power wall | والضجيج والحرارة التردد

Higher clock needs higher voltage, and dynamic power is about $C \cdot V^2 \cdot f$, so power grows roughly with f^3. More power means more heat, which needs bigger fans and heat sinks, which means more **noise**. At some point the chip can't be cooled: this is the **power wall**.

تردد أعلى يتطلب جهداً أعلى، والطاقة$C \cdot V^2 \cdot f$ ≈ ، فتزداد تقريباً مع .f^3 طاقة أكثر = حرارة أكثر = مراوح أكبر = **ضجيج** أكثر .عند حد معين لا يمكن تبريد الشريحة :هذا هو **جدار الطاقة**.

5. Clock, memory frequency, FSB, memory wall | FSBو الذاكرة وتردد التردد

In older systems, CPU clock = FSB frequency × multiplier, and memory ran at a speed tied to the FSB. The CPU got faster by raising the multiplier, but the FSB and RAM did not keep up. So the CPU often waits for memory: this is the **memory wall**.

في الأنظمة القديمة :تردد المعالج = تردد × FSB المضاعف، وتردد الذاكرة مرتبط بالـ .FSB المعالج تسارع برفع المضاعف لكن FSB والذاكرة لم يواكبا، فينتظر المعالج الذاكرة :هذا **جدار الذاكرة**.

6. FSB and QPI | QPIو الأمامي الناقل

FSB is a shared bus between the CPU(s) and the northbridge (memory controller). All cores and devices compete for it, so it becomes a bandwidth bottleneck. **QPI** (Intel, 2008) replaces it with fast **point-to-point** links, and the memory controller moves inside the CPU. This gives much higher bandwidth and lower latency and lets sockets connect directly (which leads to NUMA). It **reduces** the memory wall but doesn't remove it.

FSBناقل مشترك بين المعالج والـ) northbridge متحكم الذاكرة(، تتنافس عليه كل الأنوية والأجهزة فيصبح عنق زجاجة) **QPI** .إنتل (2008 يستبدله بوصلات **نقطة-لنقطة** سريعة وينتقل متحكم الذاكرة إلى داخل المعالج، فيزيد عرض النطاق ويقل التأخير ويسمح بربط المعالجات مباشرة)وهذا يقود إلى .(NUMA يخفف جدار الذاكرة لكنه لا يلغيه.

7. Is the 8086 limit still true? | صحيحاً؟ زال ما هل

No. Modern single-core CPUs are **superscalar and pipelined**: several ALUs, an FPU and load/store units, issuing multiple instructions per cycle with out-of-order execution. Examples: **Pentium Pro/Pentium III**, **Pentium 4** (also had Hyper-Threading), **Athlon 64** (3 integer ALUs). Parallelism exists inside one core, although it still looks like one instruction stream (SISD).

لا. المعالجات أحادية النواة الحديثة **superscalar ومُنَفَّقة (pipelined)**: عدة ALU ووحدة FPU ووحدات تحميل/تخزين، وتصدر عدة تعليمات في الدورة مع التنفيذ خارج الترتيب. أمثلة: **Pentium Pro/III** و **Pentium 4** مع Hyper-Threading و **Athlon 64** (3 وحدات ALU صحيحة). (التوازي موجود داخل النواة الواحدة رغم أنها تبدو تدفق تعليمات واحداً.(SISD)

8. ILP and the ILP wall | التعليمات مستوى توازي

ILP = running several independent instructions from one thread at once (pipelining, superscalar, out-of-order, branch prediction). The **ILP wall**: real programs have dependencies and branches, so beyond roughly 2-4 instructions per cycle, more hardware gives almost nothing. **Link to the memory wall:** a cache miss costs hundreds of cycles, so the wide pipeline sits idle waiting for data, and ILP can't hide that.

ILP = تنفيذ عدة تعليمات مستقلة من خيط واحد معاً (pipelining، superscalar، خارج الترتيب، توقع التفرع). **جدار ILP**: البرامج الحقيقية فيها اعتماديات وتفرعات، وبعد حوالي 2-4 تعليمات/دورة لا يفيد عتاد أكثر. **العلاقة بجدار الذاكرة:** ضياع الـ cache يكلّف مئات الدورات، فيبقى الـ pipeline العريض خاملاً ينتظر البيانات ولا يستطيع ILP إخفاء ذلك.

9. Three walls | الثلاثة الجدران

Power, memory, ILP (see Part A). The most important is the **power wall**.

جدار الطاقة وجدار الذاكرة وجدار ILP (انظر الجزء أ). (الأهم هو **جدار الطاقة**.

Exercise 2: Basic performance metrics

1. Wall-clock, latency, throughput

Wall-clock time: total real elapsed time. **Latency:** time for one operation or request. **Throughput:** operations per second. Example: a pizza takes 10 min (latency), an oven makes 6 per hour (throughput).

زمن الجدار: الزمن الفعلي الكلي. **الكمون:** زمن عملية واحدة. **الإنتاجية:** عدد العمليات في الثانية. مثال: البيتزا تحتاج 10 دقائق) كمون (والفرن ينتج 6 في الساعة) إنتاجية.(

2. T1 = 120 s, T = 8, T8 = 30 s

- **a.** S = 120 / 30 = **4**
  - أ. التسريع = **4**
- **b.** E = 4 / 8 = **0.5 = 50%**. Each worker is effectively used only half the time. The rest is lost to serial work, communication and waiting.
  - **ب.** الكفاءة = **50%**. كل معالج مستغل فعلياً بنصف الوقت فقط، والباقي يضيع في الجزء التسلسلي والاتصال والانتظار.

3. Unit | الوحدة

FLOP/s. Giga = 10^9, Tera = 10^{12}, Peta = 10^{15} (and Exa = 10^{18}).

FLOP/s. جيغا = 10^9، تيرا = 10^{12}، بيتا = 10^{15} وإكسا = 10^{18}).

4. "With 8 cores, will my program run 8 times faster?"

No, 8× is only the ideal upper bound. Serial parts (Amdahl), synchronization, communication and memory bandwidth make it lower.

لا، 8 أضعاف هو الحد الأعلى المثالي فقط. الأجزاء التسلسلية والمزامنة والاتصال وعرض نطاق الذاكرة تجعله أقل.

Exercise 3: Amdahl's Law

1. Which formula is correct?

Both are correct, because P = 1 − S, so (1 − P) = S. They are the same formula written differently. Note the symbol S means the serial fraction here, not the speedup.

كلاهما صحيح لأن P = 1 − S فيكون (1 − P) = S. هما نفس القانون بصيغتين. انتبه: الرمز S هنا هو الجزء التسلسلي لا التسريع.

2. s = 0.15, P = 0.85

Question	Calculation	Result
1. N = 4	1/(0.15 + 0.85/4) = 1/0.3625	**2.76**
2. N = 32	1/(0.15 + 0.85/32) = 1/0.17656	**5.66**
3. N → ∞	1/0.15	**6.67**

Question	Calculation	Result
4. Efficiency N = 4	2.76/4	**69%**
4. Efficiency N = 32	5.66/32	**17.7%**
5. Speedup 5	0.15 + 0.85/N = 0.2, so N = 0.85/0.05	**N = 17**

- **4. Observation:** efficiency drops sharply as N grows, so most added processors are wasted.
  - الكفاءة تنخفض بشدة مع زيادةN ، فمعظم المعالجات الإضافية تضيع.
- **5.** Check: 1/(0.15 + 0.05) = 5. Speedup 5 needs 17 processors, and 6.67 is never reached.
  - التسريع 5 يحتاج 17 معالجاً، والحد 6.67 لا يُبلغ أبداً.
- **6. Conclusion:** the serial fraction sets a hard ceiling (1/s) and returns diminish quickly. Reduce the serial part before buying more hardware.
  - الجزء التسلسلي يضع سقفاً صلباً (1/s) والعائد يتناقص بسرعة .قلّل الجزء التسلسلي قبل شراء عتاد أكثر.
- **7. Pessimistic:** it assumes a fixed problem size, but in practice people enlarge the problem when they get more processors.
  - يفترض حجماً ثابتاً للمسألة، بينما نكبّر المسألة عملياً عند توفر معالجات أكثر.

Exercise 4: Gustafson's Law

1. Which formula is correct?

Both are correct and equivalent (shown in 2.1).

كلاهما صحيح ومتكافئان) انظر.(2.1

2.1 Proof | البرهان (using S + P = 1)

$$S + P \cdot N = S + (1-S) \cdot N = S + N - S \cdot N = N - S \cdot (N-1)$$

2.2 Speedup

- N = 16: 16 − 0.1×15 = **14.5**
  - N = 16: **14.5**عند
- N = 64: 64 − 0.1×63 = **57.7**
  - N = 64: **57.7**عند

2.3 Comparison with Amdahl | المقارنة

N	Amdahl	Gustafson
16	1/(0.1 + 0.9/16) = **6.4**	**14.5**
64	1/(0.1 + 0.9/64) = **8.77**	**57.7**
∞	max **10**	unbounded بلا حد

2.4 Implications | الدلالات

Amdahl (fixed size): the serial part is a hard ceiling, which is useful for judging legacy code. **Gustafson** (scaled size): with a bigger problem the parallel part grows, so speedup stays almost linear, which is useful for big systems (weather, AI training).

أمدال حجم ثابت :(الجزء التسلسلي سقف صلب، ومفيد لتقييم الكود القديم .**غوستافسون** حجم متزايد :(مع مسألة أكبر يكبر الجزء المتوازي فيبقى التسريع شبه خطي، ومفيد للأنظمة الكبيرة) الطقس، تدريب الذكاء الاصطناعي.(

Exercise 5: Amdahl vs Gustafson (N = 16, s = 0.15)

- **1.** Gustafson: 16 − 0.15×15 = **13.75**
  - **1.**غوستافسون **13.75** :

- **2.** Amdahl: 1/(0.15 + 0.85/16) = 1/0.2031 = **4.92**. Gustafson is much higher.
  - **2.**أمدال .**4.92** :غوستافسون أعلى بكثير.
- **3. Key difference:** Amdahl keeps the problem size fixed (same work, done faster). Gustafson scales the problem with N (more work in the same time).
  - **3.**أمدال :حجم المسألة ثابت) نفس العمل أسرع .(غوستافسون :المسألة تكبر مع) N عمل أكثر في نفس الوقت.(
- **4. No, S is measured differently.** In Amdahl, S is the serial fraction of the single-processor run. In Gustafson, S is the serial fraction of the N-processor parallel run.
  - **4.لا، S تُقاس بشكل مختلف .**عند أمدال هي نسبة الجزء التسلسلي في تنفيذ معالج واحد، وعند غوستافسون هي نسبته في التنفيذ المتوازي على N معالج.

Bonus check: converting 0.15 (parallel run) to the single-processor run gives s = 0.15/13.75 ≈ 0.0109 (1.09%). Amdahl with this s gives 1/(0.0109 + 0.9891/16) = **13.75**. Same answer, so the two laws agree once S is measured the same way.

تحقق إضافي :تحويل) 0.15 تنفيذ متوازٍ (إلى تنفيذ المعالج الواحد يعطي1.09% ≈ s ، وأمدال بهذه القيمة يعطي .13.75 نفس الجواب، فالقانونان يتفقان عند قياس S بنفس الطريقة.

Exercise 6: Flynn's taxonomy

1.1 Streams | التدفقات

- **Instruction stream:** the sequence of instructions a processor executes (the program flow, driven by a control unit).
  - **تدفق التعليمات :**تسلسل التعليمات التي ينفذها المعالج.
- **Data stream:** the sequence of data operands moving between memory and the processor.
  - **تدفق البيانات :**تسلسل المعطيات المتدفقة بين الذاكرة والمعالج.

1.2 Two least used: SISD and MISD

Common characteristic: both have a single data stream (no data parallelism).

الخاصية المشتركة :كلاهما تدفق بيانات واحد) بلا توازي بيانات.(

- **SISD:** pure sequential, essentially obsolete as multicore replaced single-core.
  - **SISD:**تسلسلي بحت وشبه منقرض بعد الأنوية المتعددة.
- **MISD:** few problems need different operations on the same single data item. It is used only in fault-tolerant redundancy (e.g. space shuttle flight computers voting).
  - **MISD:**قليلة المسائل التي تحتاج عمليات مختلفة على نفس البيانات، وتُستعمل فقط في التكرار لتحمل الأعطال.

1.3 Memory systems | الذاكرة أنظمة

Flynn classifies instruction and data streams, not memory. Shared-memory machines (multicore) and distributed-memory machines (clusters) are **both MIMD**. They differ in how memory is organized, which is a different axis.

تصنيف فلين يخص تدفقات التعليمات والبيانات لا الذاكرة .الأنظمة ذات الذاكرة المشتركة (multicore) والموزعة (clusters) **كلتاهما MIMD**، والفرق في تنظيم الذاكرة وهو محور آخر.

1.4 Parallelism types | التوازي أنواع

- **Data parallelism** → **SIMD** in hardware (GPU, vectors), and also SPMD (same program, different data) on MIMD machines such as MPI.
  - **توازي البيانات SIMD** ← في العتاد، وكذلك SPMD على أجهزة MIMD مثل.MPI
- **Task parallelism** → **MIMD** (different tasks, different data).
  - **توازي المهام.MIMD** ←
- **Pipeline parallelism** → MIMD in practice (stages run as different tasks).
  - **توازي خط الأنابيب** MIMD ← عملياً.

2. Flynn's 1972 SIMD subcategories | الفرعية الفئات

Type	English	العربية	Modern example
Array processor	Many processing elements, each with its own memory and registers; one control unit broadcasts the instruction.	عناصر معالجة كثيرة لكل منها ذاكرة وسجلات وجهاز تحكم واحد يبث التعليمة.	**GPU** (now SIMT)
Pipelined (vector) processor	Reads vectors from a central register/memory, streams them through pipelined units, writes back.	يقرأ متجهات من مصدر مركزي ويمررها عبر وحدات pipeline ثم يكتب النتيجة.	**AVX/SSE** units, ARM SVE, NEC SX-Aurora
Associative processor	Broadcasts an instruction plus a value; each unit compares its local data and decides independently.	يبث تعليمة وقيمة وكل وحدة تقارن بياناتها وتقرر باستقلال.	**CAM/TCAM** in network routers, mask/predicate execution in AVX-512

3. SIMD vs SIMT

See Part A. Example of SIMT: **NVIDIA GPUs (CUDA)**, also AMD wavefronts.

انظر الجزء أ .مثال :SIMT **معالجات NVIDIA (CUDA).**

4. Classification | التصنيف

System	Category	Why
Single-core CPU, one ALU + one FPU	**SISD**	One instruction stream, one data stream
GPU, thousands of cores, same instruction	**SIMD (SIMT)**	Same instruction, different data
Cluster of multicore CPUs, different tasks	**MIMD**	Independent instructions and data
Vector processor, same op in a pipeline	**SIMD**	Pipelined/vector processor
Multicore CPU, threads with different instructions	**MIMD**	Different instructions, different data

Exercise 7: Categories of HPC systems

1. Differences | الفروقات

- **Shared memory:** cores share one address space and communicate via shared variables.
  - **ذاكرة مشتركة :**الأنوية تتشارك فضاء عناوين واحداً وتتواصل بمتغيرات مشتركة.
- **Distributed memory:** independent nodes, each with local memory, communicating by network messages.
  - **ذاكرة موزعة :**عقد مستقلة بذاكرة محلية وتتواصل برسائل عبر الشبكة.
- **Accelerated:** CPU + GPU/TPU with separate memories; work is offloaded and data moved explicitly.
  - **مسرّعة :**CPU مع GPU/TPU بذاكرات منفصلة ونقل صريح للبيانات.

2. SMP vs NUMA | الفرق

- **SMP:** uniform access. All processors reach one memory with the same latency through a common bus. It is simple but the bus limits scaling.
  - **SMP:**وصول موحّد :كل المعالجات تصل لذاكرة واحدة بنفس الكمون عبر ناقل مشترك، بسيط لكن الناقل يحدّ التوسع.
- **NUMA:** memory is split into regions, each attached to a socket. Every processor can access all memory, but local is fast and remote is slower (through QPI/UPI).

- **NUMA:** الذاكرة مقسمة لمناطق مرتبطة بكل معالج، والكل يصل للكل لكن المحلي سريع والبعيد أبطأ.
- **Why placement matters:** if a thread works on data in another socket's memory, every access pays extra latency and uses interconnect bandwidth. Place data near the thread that uses it (e.g. "first-touch" allocation, thread pinning).
- **لماذا يهم التوزيع:** إذا عمل خيط على بيانات في ذاكرة معالج آخر فكل وصول يكلف كموناً إضافياً ويستهلك عرض نطاق الوصلة. ضع البيانات قرب الخيط الذي يستعملها.

3. Classification | التصنيف

- (a) Laptop with 8-core CPU → **Shared memory**
- أ) (حاسوب محمول بمعالج 8 أنوية ← **ذاكرة مشتركة**)
- (b) Three servers linked by a network → **Distributed memory**
- ب) (ثلاثة خوادم متصلة بشبكة ← **ذاكرة موزعة**)
- (c) Workstation with CPU + GPU over PCIe → **Accelerated**
- ج) (محطة عمل بـ CPU و GPU عبر PCIe ← **مسرّعة**)

4. GPU-accelerated node | مسرّعة عقدة

(a) Node level: **MIMD**. (b) Inside the GPU: **SIMD (SIMT)**.

أ) (على مستوى العقدة) **MIMD**. ب) (داخل الـ GPU: **SIMD (SIMT)**.

Exercise 8: Programming models

Model	System category	Data exchange
OpenMP	Shared memory	Implicitly through **shared variables** (with barriers/critical for sync)
MPI	Distributed memory	Explicit **messages** (send/recv, collectives)
CUDA / OpenCL	Accelerated	Explicit **host** ↔ **device copies** over PCIe/NVLink (e.g. cudaMemcpy)
Pthreads	Shared memory	**Shared variables** protected by mutexes/condition variables

Exercise 9: Anatomy of a supercomputer (1000 nodes × 2 CPU sockets × 4 GPUs)

1. Layers and memory | الطبقات والذاكرة

Layer	What	Memory model
Cluster	1000 nodes on a fast network (e.g. InfiniBand)	**Distributed** (each node has its own memory)
Node	2 sockets + 4 GPUs (so 2000 CPUs and 4000 GPUs in total)	CPU RAM is **shared/NUMA** between the 2 sockets; GPUs have their own memory (**accelerated**)
Core	Cores inside each CPU, each with SIMD (AVX) units	**Shared** memory and caches
Accelerator	Each GPU: thousands of cores	Separate **device memory** (HBM), explicit copies

2. Programming model per layer | نموذج لكل البرمجة طبقة

- Cluster → **MPI**
- العنقود **MPI** ←
- Node/cores → **OpenMP** (or Pthreads), NUMA-aware, plus **SIMD** vectorization inside the core

- العقدة/الأنوية) **OpenMP** ← أو (Pthreads مع مراعاةNUMA ، إضافة إلى التوجيه **SIMD** داخل النواة
- GPU → **CUDA / OpenCL / HIP / SYCL**
- الـ**CUDA / OpenCL / HIP / SYCL** ← GPU
- In practice it is **hybrid: MPI + OpenMP + CUDA**, often one MPI process per GPU.
- عملياً هو **هجين: MPI + OpenMP + CUDA**، وغالباً عملية MPI لكل GPU.

3. "A MIMD machine with SIMD inside" | لماذا؟

At the top, thousands of cores and nodes run **different instruction streams on different data**, which is **MIMD**. Inside each core (vector units) and each GPU (SIMT warps), **one instruction works on many data items**, which is **SIMD**. So it is MIMD on the outside and SIMD on the inside.

في الأعلى آلاف الأنوية والعقد تنفّذ **تعليمات مختلفة على بيانات مختلفة** وهذا **MIMD**. وداخل كل نواة) وحدات المتجهات (وكل GPU (warps)**تعليمة واحدة على بيانات كثيرة** وهذا **SIMD**. فهي MIMD من الخارج و SIMDمن الداخل.

Good luck! | !بالتوفيق